



**VAAAL UNIVERSITY
OF TECHNOLOGY**

Inspiring thought. Shaping talent.

**DESIGN AND IMPLEMENTATION OF ESP32 BASED
SMART IRRIGATION MONITORING AND CONTROL
SYSTEM FOR EFFICIENT SOIL WATER
MANAGEMENT SYSTEM IN FARMS**

Diploma in Electrical Engineering

Department of Electrical Engineering

Faculty of Engineering and Technology

Name & Surname:
MPHO NGWENYA

Student Number:
240548531

Diploma in Electrical Engineering

Supervisor: Mr D.F OJO-SERIKI

Date: 21 May 2026

Declaration

I MPHO NGWENYA, declare that this report is my original work and that it has not been presented to any other university or institution for similar or any other degree award. It is being submitted for Electrical Engineering Projects 4 (EEWPR4A) to the Department of Electrical Engineering at the Vaal University of Technology, Vanderbijlpark.

NGWENYA MB
Signature

14/04/2026
Date

Acknowledgement

Student Number	Name and Surname
240548531	NGWENYA MPHO
223402087	BAPELA NTLHWANE
240880005	MATHABATHA MOKITI
223806358	MSENGI NDLANDLAMUKA
224256408	SABELA LULAMA

Dedication

I dedicate this project to myself, friends and family. Thank you for your support and believing in me.

Table of Contents

Chapter 1: Introduction and Purpose of the study	8
1.1 Introduction	9
1.2 Background	9
1.3 Problem Statement.....	10
1.4 Significance of the study	11
1.5 Objectives.....	11
1.6 Project Plan	12
1.7 Proposed Budget	13
1.8 Time Schedule	14
1.9 Limitations.....	14
1.10 Conclusion	15
Chapter 2: Research Design and Methodology	16
2.1 Introduction.....	16
2.2 Scope of work	17
2.2.1 Microcontroller	17
2.2.2 Sensors	17
2.2.3 Status Indicators	20
2.2.4 Web Server	20
2.2.5 Database.....	21
2.3 Conclusion	21
Chapter 3: Hardware Design	22
3.1 Introduction.....	22
3.2 Hardware Design	22
3.2.1 Electronics Circuit Designs	23
3.2.1.1 Ultrasonic Water Level Sensor	23
3.2.1.2 Soil Moisture Sensor	23
3.2.1.3 PIR Motion Sensor.....	24
3.2.1.4 DHT22 Temperature and Humidity Sensor	24
3.2.1.5 Rainfall Sensor Circuit Design.....	25
3.3 Main Circuit Design	25
3.4 Wiring	26

3.5	Enclosure Design	27
3.6	Safety Precautions.....	28
3.7	Conclusion	29
Chapter 4: Software Design of the Design and Implementation of ESP32 based Smart Irrigation Monitoring and Control System for Effective Soil-Water System in Farms .		30
4.1	Introduction.....	30
4.2	Software Design.....	30
4.2.1.	Use Case Analysis	30
4.2.1.1	Activity Diagram – Execute Irrigation Logic Use Case	31
4.2.1.2	Sequence Diagram.....	32
4.2.2.	Flask Server	33
4.2.2.1	Wi-Fi Connectivity and Data Transfer	33
4.2.2.2	Database Connection and Communication	34
4.2.3.	Database Modelling	35
4.2.4.	Web Interface	37
4.2.4.1	Home Page	37
4.2.4.2	Login Page	37
4.3	Conclusion	38
Chapter 5: Results and Discussions.....		39
5.1	Introduction.....	39
5.2	Findings.....	39
5.3	Results	39
5.3.1	Sensor Data.....	39
5.2.1	Administrator Results	40
Chapter 6: Conclusion and Recommendations.....		42
6.1	Conclusion	42
6.2	Recommendations	42
References		43

List of Tables

Table 1: The Budget Description of The Smart Irrigation Monitoring and Control System	13
Table 2: Project Time Schedule	14
Table 3: Sensor to ESP32 Connection	27

List of Figures

Figure 1: System Block Diagram	16
Figure 2: ESP32 Microcontroller (Fity.club, 2026)	17
Figure 3: Soil Moisture Sensor	17
Figure 4: DHT22 Temperature and Humidity Sensor (Liu, n.d.)	18
Figure 5: Rainfall Sensor (Anon, n.d.)	18
Figure 6: Ultrasonic Sensor (ElecFreaks, 2011)	19
Figure 7: PIR Motion Sensor (Handson Technology User Guide PIR Motion Sensor, n.d.)	19
Figure 8: LED (Hussain and Wani, 2024)	20
Figure 9: Python Flask Web Server	20
Figure 10: MySQL Database	21
Figure 11: Hardware Design Block Diagram	22
Figure 12: Ultrasonic Sensor	23
Figure 13: Soil Moisture Sensor Circuit Design	23
Figure 14: PIR Motion Sensor Circuit Design	24
Figure 15: DHT22 Temperature and Humidity Sensor Circuit Design	24
Figure 16: Rainfall Sensor Circuit Design	25
Figure 17: Main Circuit Design Sketch	26
Figure 18: System Wiring	27
Figure 19: Enclosure Design	28
Figure 20: Software Design Block Diagram	30
Figure 21: Use Case Diagram	31
Figure 22: Activity Diagram	32
Figure 23: Sequence Diagram	33
Figure 24: ESP32 Wi-Fi Setup	34
Figure 25: Capturing ESP32 Sensor Data	34
Figure 26: Flask and Database Connection	35
Figure 27: Sending to Database	35
Figure 28: Physical Database Model	36
Figure 29: Home Page	37
Figure 30: Administrator Login Page	37
Figure 31: Farmer Login Page	38
Figure 32: Water Level Readings	40
Figure 33: Water Level Database Values	40
Figure 34: Administrator Dashboard	41
Figure 35: Sensor Data from Administrator View	41

Acronym

IoT	Internet of Thing
ESP32	Espressif 32-bit Microcontroller
PIR	Passive Infrared Sensor
Wi-Fi	Wireless Fidelity
LED	Light Emitting Diode

Chapter 1: Introduction and Purpose of the study

1.1 Introduction

The economy of countries across the African continent has a high reliance on agriculture, playing a huge role in job creation, ensuring food security, supporting industrial development and reducing poverty. It consists of factors that sustain it which includes crop production, climate change, water management, and soil irrigation. Agriculture has developed significantly in recent years, through technology advancement that shape modern farming. This study focuses on developing a smart irrigation and monitoring system for soil-water management in farms. It is designed to address gaps such as the lack of real-time monitoring of soil, climate conditions, the inefficient use of water resources for irrigation.

1.2 Background

Agriculture is one of the key industries that drives economic growth, supports food security, and sustains both human and animal life. It is defined as the science and practice of cultivating the soil, growing crops, and raising livestock to produce food and other essential products (Harris and Fuller, 2014). The global population continues to increase, directly threatening food security. Therefore, the need for advanced technology to sustain food production has become crucial. In the past, simple tools made from wood were used to cultivate the land, but these methods are no longer sufficient to meet the demands of a rapidly growing population. Today, modern technology is required to support large-scale agricultural production.

Agriculture plays a significant role in the global economy, contributing substantially to Gross Domestic Product (GDP) and creating employment opportunities, particularly in developing countries (A. Francisco, 2019). It provides jobs across various sectors, including farming, transportation, processing, packaging, and marketing of agricultural products. These employment opportunities help families in rural communities earn a living and contribute to the development of local infrastructure. In addition, agricultural production supports international trade, allowing countries to earn foreign exchange and promote economic development.

As agriculture continues to expand, farmers face several challenges that affect crop production and livestock management. One of the major challenges faced today is the lack of adequate information technology to enable farmers to gather real-time data. This data is essential for identifying factors affecting production and for suggesting improved farming practices to workers (Fan, 2021). Without access to such technology, farmers may suffer financial losses because they invest in improvements without fully understanding the root causes of their production issues. Furthermore, limited access to information technology reduces awareness and adoption of modern systems, such as IoT-based solutions, causing farmers to rely on outdated and less efficient methods.

This project focuses on designing a modern farming system that addresses farmers lack of information technology and automates irrigation based on data collected from the farm. It enables farmers to access real-time data and monitor environmental conditions using IoT

systems, while also automating the irrigation process. The main objective of the project is to improve soil–water management and reduce inefficient water usage in farms. This is achieved by capturing environmental data from the field and using it to support informed irrigation decisions. Overall, the project is data-driven and aims to bridge the gap caused by outdated farming methods and limited access to information technology.

The management of a farm relies on several environmental conditions that must be continuously observed to support effective decision-making. Soil-moisture plays a vital role in determining the irrigation needs of crops, enabling farmers to apply water at the right time and in the right amount. Natural contributors such as rainfall also influence soil-moisture levels and assist in planning appropriate irrigation schedules. In addition, soil-moisture is affected by evaporation, which depends on the ambient temperature and humidity of the environment. Under high temperatures and low humidity, the soil loses water rapidly, making it important to monitor these conditions to maintain healthy crop growth. Monitoring overall water usage and the water level in irrigation tanks further supports efficient water management and promotes sustainable farming practices.

In 2023, Gilroy P. Pereira, together with Mohamed Z. Chaari and Fawwad Daroge, developed and implemented an IoT-enabled smart drip irrigation system using the ESP32 microcontroller (Pereira, Chaari and Daroge, 2023). The system was designed using multiple sensors to capture environmental conditions such as soil moisture, temperature, humidity, and water flow. For monitoring and control the Blynk mobile application was used, which connects via the ESP32's built-in Wi-Fi to allow users to access real-time sensor data, visualize environmental changes, and manually override the system when necessary. Pereira and his colleagues contributed significantly to advancing the integration of technology in modern agriculture, particularly in improving irrigation practices and reducing manual labour by automating irrigation processes based on real-time environmental conditions.

Nowadays, technology is evolving towards the use of artificial intelligence in predicting, evaluating, and managing complex systems. In agriculture, artificial intelligence plays an important role in assisting farmers to make smart irrigation and farming decisions. Ravi Ray Chaudhari, alongside his colleagues, developed a hybrid framework for precise farming using Machine Learning (ML), IoT sensors, GPS/GIS technologies and data analytics tools (Chaudhar R, 2023). The sensors were used to collect real-time environmental data from the farm, including soil conditions, weather parameters, and crop health information. This data was processed using machine learning algorithms to generate accurate predictions. This approach improved efficiency and significantly reduced the waste of resources.

1.3 Problem Statement

Farming faces a significant challenge due to the use of traditional irrigation methods, which rely on manual, imprecise watering. The integration of modern technology on farms has not been fully developed, resulting in a lack of real-time monitoring of environmental conditions needed for effective irrigation. As a result, irrigation is often poorly managed, leading to water wastage through over-irrigation and, in the long term, reduced crop productivity. This problem is most visible in rural and semi-urban farming areas, particularly in Africa, where access to

advanced farming technologies is limited. This problem becomes more critical during dry seasons and irregular weather conditions, when proper irrigation management is critical for crop growth. This issue is important because it threatens food security, increases farming costs, and contributes to the depletion of water resources.

To address this challenge, this project proposes a smart irrigation monitoring and control system. The system uses a microcontroller integrated with IoT sensors to collect environmental data and automate irrigation decisions based on programmed conditions. It is designed for use on farms in rural and semi-urban areas and can support both small-scale and large-scale farmers in African communities. The system operates through real-time monitoring, allowing farmers to detect water shortages, especially during dry seasons, and irrigate crops accurately when needed. This solution is important because it improves crop productivity, reduces water wastage, and saves time and labour, while promoting more efficient and sustainable farming practices.

1.4 Significance of the study

The study on the design and implementation of an ESP32-based smart irrigation monitoring and control system is primarily to monitor environmental conditions and control the irrigation process scheduling. Since traditional irrigation systems still rely heavily on manual decision-making and fixed watering schedules, this system is significant and needed because irrigation decisions are taken based on real-time data of the farm environmental conditions.

The purpose of this study is:

- Improved water use efficiency in farms
- Control irrigation schedules based on real-time environmental data.
- Reduce manual labour and costs for monitoring through remote web-based monitoring and control system.
- Promote the adoption of technology for improved agricultural practises.
- Reduce manual intervention and human error.

Overall, this study contributes to the advancement of smart agriculture technologies and supports sustainable, efficient, and productive farming systems.

1.5 Objectives

The objectives of this study are tailored to guide the development of an ESP32 based smart irrigation monitoring and control system. They provide a pathway for the research and outline the key stages of the system development.

The objective of this research are:

- To design a system that captures environmental data in the farm.
- To implement a programmed logic that makes irrigation decision.
- To automated the irrigation process in the farm.
- To develop a web-based server to monitor and control the system in real-time.
- To improve the water efficiency in farms.

1.6 Project Plan

This section outlines the steps that will be followed to design, develop, implement, and establish the smart irrigation system. The plan ensures a structured approach to achieving the objectives of the study.

Objective 1: Designing the System

Estimated time: 1-2 weeks

- Identify and select the components required for the system.
- Simulate the system to identify the behaviour of each component.
- Design the system architecture showing how hardware and software components interact.
- Define the data flow from sensor data collection to the final control action.

Objective 2: Implementing Programmed Logic

Estimated time: 2-3 weeks

- Read the environmental data captured by each sensor.
- Use conditioning and thresholds to implement the logic.
- Trigger output based on the logic implemented.

Objective 3: Automating of Irrigation

Estimated time: 1-2 weeks

- Send a trigger to the processing unit(microcontroller) notifying that irrigation conditions are met.
- Alert the water pump to irrigate automate.
- Send the irrigation trigger to the status indicator (LED, buzzer).

Objective 4: Developing a Web-based Server

Estimated time: 1-2 weeks

- Establish Wi-Fi connectivity between microcontroller and server.
- Send the farm environmental conditions captured by sensors to the server.
- Display environmental conditions in real-time in the website.

The expected output is a fully functional ESP32-based smart irrigation monitoring and control system, along with documented analysis of its performance and efficiency.

1.7 Proposed Budget

Table 1: The Budget Description of The Smart Irrigation Monitoring and Control System

Description	Cost (ZAR)
ESP32 Microcontroller	125.00
Breadboard	25.00
Wires	32.00
Ultrasonic Sensor	38.00
Rainfall Sensor Module	35.00
DHT22	54.00
Soil Moisture Sensor	25.00
Total	334.00

1.8 Time Schedule

Table 2: Project Time Schedule

Activity	Jan				Feb				Mar				Apr				May			
	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4	Week 1	Week 2	Week 3	Week 4
Title Definition																				
Proposal Writing																				
Literature Review																				
Component Quotation																				
Simulation																				
Parameter Testing																				
Design Concept																				
Chapter 2 writing																				
Chapter2 submission																				
Chapter 3 writing																				
Chapter3 submission																				
Chapter 4 writing																				
Chapter4 submission																				
Chapter 5 writing																				
Chapter5 submission																				
Chapter 6 writing																				
Chapter6 submission																				
Demonstration																				
Presentation																				
Final Submission																				

1.9 Limitations

The development of the Design and Implementation of ESP32 based Smart Irrigation Monitoring and Control System for Effective Soil-Water System has several limitations that effect the overall outcome of the project. These limitations include:

- The Wi-Fi signal from client to server may be interrupted effecting data transmission.
- The microcontroller depends on power supply from a PC without any backup power.

- The timeframe to assess the system performance over a long period is limited.
- Data is stored in a local SQL database with no recovery or backups for hard drive failure.
- The system reacts based on real-time data with no predictive capabilities.

Despite these limitations, the study remains feasible and valuable for demonstrating the potential of smart irrigation technologies in improving agricultural efficiency.

1.10 Conclusion

In this chapter the foundation overview of the study was discussed by outlining the importance of agriculture, challenges associated with outdated traditional agricultural methods and the significance of adopting innovative technologies to improve crop production and the efficient use of water. The chapter highlighted the significance of integrating a microcontroller with IoT sensors to monitor real-time environmental conditions in farms. Furthermore, the objectives of the project were clearly defined, demonstrating how each stage contributes to solving the main agricultural problem along with the budget plan and the time schedule required to complete the project. The next chapter focuses on project design and the overall architecture by outlining hardware and software components to be implemented and how they interact with each other.

Chapter 2: Research Design and Methodology

2.1 Introduction

The design of this project follows a structured engineering method that includes planning, experimentation, and prototyping to achieve a fully functional smart irrigation system. The system is monitored and controlled by an ESP32 microcontroller integrated with IoT sensors that capture real-time environmental conditions. The system relies on continuous data collection to automate irrigation accurately and efficiently. Figure 1 presents the block diagram of the system architecture, showing how the ESP32, sensors, and irrigation components operate together within the design.

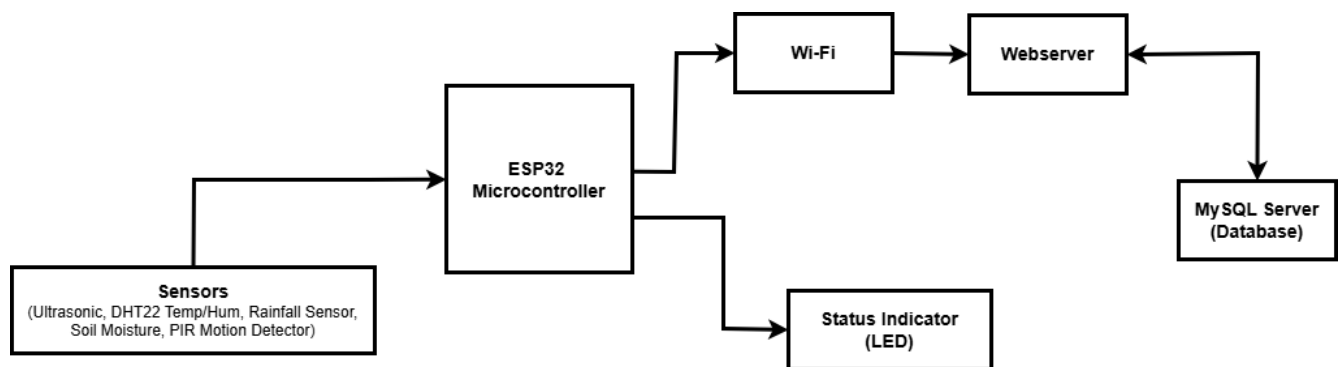


Figure 1: System Block Diagram

The development of the ESP32-based automatic irrigation system follows the steps below:

- Learning and understanding the required hardware components for the system.
- Designing the system architecture and defining the flow of data from the sensors.
- After establishing the data flow, the sensors are connected and tested through simulations before physical integration with the ESP32.
- The logic used to automate irrigation is programmed based on thresholds determined by environmental conditions.
- Wi-Fi connectivity is introduced to allow the ESP32 to transmit data through framework to the database and web-server for real-time monitoring and control.
- Final testing of the complete system is carried out in a small-scale farming environment to evaluate performance and reliability.

Once the smart irrigation system is fully assembled with all sensors integrated and the programmed logic implemented, the final outcome is a functional automated irrigation system capable of monitoring and controlling irrigation processes. The system is expected to improve water usage, enhance crop productivity, support better decision-making for farmers, and contribute to sustainable food production

2.2 Scope of work

2.2.1 Microcontroller

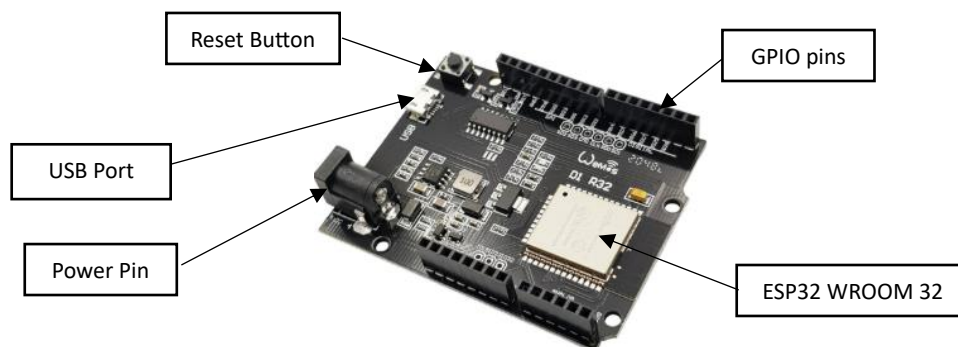


Figure 2: ESP32 Microcontroller (Fity.club, 2026)

A microcontroller is a device that serves as a central processing unit with different types of components integrated within a single chip (G. Gridling, 2007). There are a variety of microcontroller present today including Arduino Uno, Raspberry Pi, NodeMCU ESP8266, and STM32. For the design of the smart irrigation monitoring and control system, the ESP32 microcontroller will be implemented. The ESP32 is low-cost, low-power development board with build-in features including dual-core CPU, digital and analog GPIO pins, DAC/ADC pins, and 3.3V and 5V supply pins, making it suitable for IoT applications. What makes it more suitable for this design is its Wi-Fi and Bluetooth capabilities, which allow it to wirelessly communicate with other devices. For this system five ESP32 microcontrollers are used. Each ESP32 serves a purpose of reading sensor signals in real-time through its GPIO pins, and transmits the data to the Flask server by connecting to the PC's Wi-Fi access point.

2.2.2 Sensors

2.2.2.1 Soil Sensor

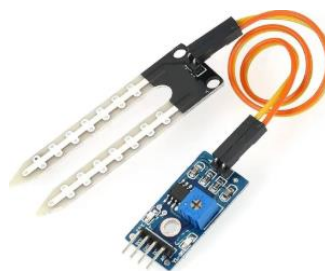


Figure 3: Soil Moisture Sensor

The soil moisture sensor measures the amount of water present in the soil. The water content in soil can be measured using a variety of devices including a capacitive soil moisture sensor, gypsum block sensor, tensiometer and Time Domain Reflectometry (TDR) sensor. For this project the resistive soil moisture sensor will be implemented. The resistive soil moisture sensor is best suited for the project requirements because it is low-cost, simple to integrate with ESP32, unlike the capacitive sensor which is expensive, and TDR sensor which is complex. It works using two conductive probes that detect changes in resistance based on soil moisture

level (S. Chowdhury, 2022). These variations are converted into voltages and mapped into percentage values. This sensor is essential in the project because it triggers irrigation based on programmed threshold percentages.

2.2.2.2 Temperature and Humidity Sensor

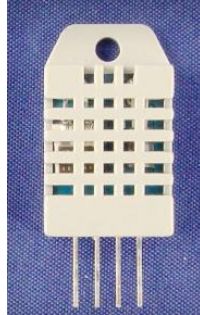


Figure 4: DHT22 Temperature and Humidity Sensor (Liu, n.d.)

Measuring temperature and humidity in farms important to allow farmers to monitor environmental conditions which impact irrigation through evaporation. A variety of sensors can be used to measure the ambient temperature and humidity which includes the DHT11, BME280, and the SHT31. The most suited temperature and humidity sensor for the requirements of this project is the DHT22 sensor. It is capable of measuring both ambient temperature and humidity in a single sensor. It has low-power consumption, reliable precision and can be easily integrated with the ESP32 using the DHT.h library in Arduino IDE. The sensor uses a capacitive humidity sensor to detect changes in moisture levels in the air, and an NTC thermistor to measure temperature based on variations in electrical resistance (Intan Kusuma Wardani et al., 2023) The sensor sends digital readings to the ESP32 using a single digital pin, which are used to monitor evaporation rate and the atmospheric moisture level in the farm.

2.2.2.3 Rainfall Sensor



Figure 5: Rainfall Sensor (Anon, n.d.)

The rainfall sensor is responsible for measuring the presence of rainfall, which is an essential to managing the irrigation process. It provides rainfall data that helps to avoid the inefficient use of water and over-irrigation in farms. There are a variety of devices which can be used to detect rainfall which include a tipping bucket rain gauge, optical rain sensor, and the weather station module. For this project a low-cost, simple to interface resistive rainfall sensor is implemented. This resistive rainfall sensor consists of a conduction board. When water lands

on the sensor board, it conducts electricity between these traces, causing changes in the resistance which is then converted into a voltage signal read by the ESP32 via an analog pin. The sensor ensures that irrigation does not take place during rainfall, preventing water waste and protecting crops from overwatering.

2.2.2.4 Water Level Sensor



Figure 6: Ultrasonic Sensor (ElecFreaks, 2011)

The water level sensor is a device used to monitor the water level in the tank that will be used to irrigate the farm. The sensor also helps to determine the amount of water used over a specific period of time allowing farmers to better manage water resources. There are a variety of water level sensors that can be used to perform the same function, which includes the float switch sensor, capacitive water level sensor, and the Infrared (IR) distance sensor. For the project the ultrasonic sensor will be implemented to measure and monitor the water level in the tank. The sensor works by sending ultrasonic waves from its transmitter, which bounce off the surface of the water and return as an echo to the receiver (A Djalilov et al., 2023). The time taken for the echo to return is converted into a distance measurement using the speed of sound. This measurement allows the system to monitor water availability, ensuring that the tank has enough water before activating the irrigation process.

2.2.2.5 Motion Detector



Figure 7: PIR Motion Sensor (Handson Technology User Guide PIR Motion Sensor, n.d.)

The motion sensor is a device that is used to detect the motion of humans and animals within a defined area and range. For this smart-irrigation and monitoring system, the motion sensor is implemented to detect the presence of human or farmers in the field before the automatic irrigation is triggered for safety. A variety of motion sensors can be used to detect movement, which includes microwave motion sensor, camera-based motion sensor, and the infrared beam sensor. For this project the Passive Infrared (PIR) sensor will be implanted. Compared to the other motion sensor the PIR is low-cost, low-power consumption with reliable accuracy using infrared radiation heat emitted by warm bodies, and produce a HIGH/LOW digital output based on whether the motion is detected (Sumathi et al., 2022). The PIR sensor is used to delay

irrigation to prevent water spraying onto farmers and ensures that irrigation occurs only when the farm area is clear.

2.2.3 Status Indicators



Figure 8: LED (Hussain and Wani, 2024)

A status indicator is a device or a variable that provides a visual representation of an underlying condition or state. In this project Light Emitting Diodes (LED) are implemented to indicate the operational status of each sensor. These LEDs are also used show the status of the water pump. There are a variety of status indicators available for different application which include seven segment display, Liquid Crystal Display and a buzzer. Compared to the LEDs, the LCD display uses more GPIO pins and wiring and the buzzer provides audio feedback rather than visual indication which makes these less convenient for smart-irrigation system. LEDs are low-cost, require minimal GPIO pins, have a low power consumption and simple to program using digital HIGH to turn on, and a digital LOW to turn off. The LEDs work to allow farmers to physically visualize the status of the system in the farm site without relying on the website dashboard.

2.2.4 Web Server



Figure 9: Python Flask Web Server

Flask is a lightweight framework written in Python and serves as the server-side back-end providing the functionality of object request, routing systems for endpoints and HTTP utilities (Ghimire, 2020). There are a variety of frameworks available today which includes Django, Node.js with Express, and FastAPI. Flask is implemented in this project because it supports HTTP requests, written in Python, lightweight, and does not require additional built-in features that add complexity. For this project Flask acts as the middleware between ESP32 devices,

database and web interface to provide live feedback of each sensor reading. It works by receiving ESP32 sensor data over Wi-Fi connectivity, processes the data, stores it in the database and serves the website with real-time sensor data.

2.2.5 Database



Figure 10: MySQL Database

A database is a data storage system, that stores a collection of interconnected data in a computer hardware, and used by a software to be captured, processed, visualized, and manipulated (J. Wahyudi, 2022). In this project the database is responsible for storing all sensor readings. There are a variety of databases available which include MongoDB, Microsoft Access, and PostgreSQL. For this project MySQL Database is implemented. MySQL stored data tables using rows and columns. Unlike other databases MySQL is lightweight, integrates perfectly with Flaks and Python, and uses simple query language. MySQL serves as a central data storage for the systems ensuring all sensor data is stored and retrieved safely for real-time monitoring on website dashboard.

2.3 Conclusion

This chapter presented the overall system architecture, research design and methodology used to develop the smart irrigation system. A block diagram which includes all components to be used in the system was discussed. This includes a brief description of each block diagram and the role of each block in the system and how they contribute to efficient and automated irrigation. The next chapter will focus on the implementation of the system, including the hardware setup, prototyping and testing procedures used to achieve the final working system.

Chapter 3: Hardware Design

3.1 Introduction

This chapter presents the hardware design of the ESP32-based smart irrigation monitoring and control system. It focuses on explaining how each circuit and component is designed, connected, and integrated to enable accurate environmental monitoring and automated irrigation on farms. Based on Figure 1 which shows the block diagram of the overall design describing the input-output relationship of the system. The block diagram describes the hardware architecture incorporating multiple environmental sensors. This including soil moisture, rainfall, water level, temperature, and humidity. These sensors capture real-time environmental data from the farm and transmit it to the ESP32 microcontroller, which serves as the central processing unit.

3.2 Hardware Design

The hardware design for the ESP32-based smart irrigation monitoring and control system consists of multiple hardware parts. Figure 11 below is the block diagram of the hardware system. In this section a detailed walkthrough is provided along with circuit diagrams for each circuit design to be implemented. The complete integration of each circuit should be able to perform irrigation automatically with minimal supervision for the farmers.

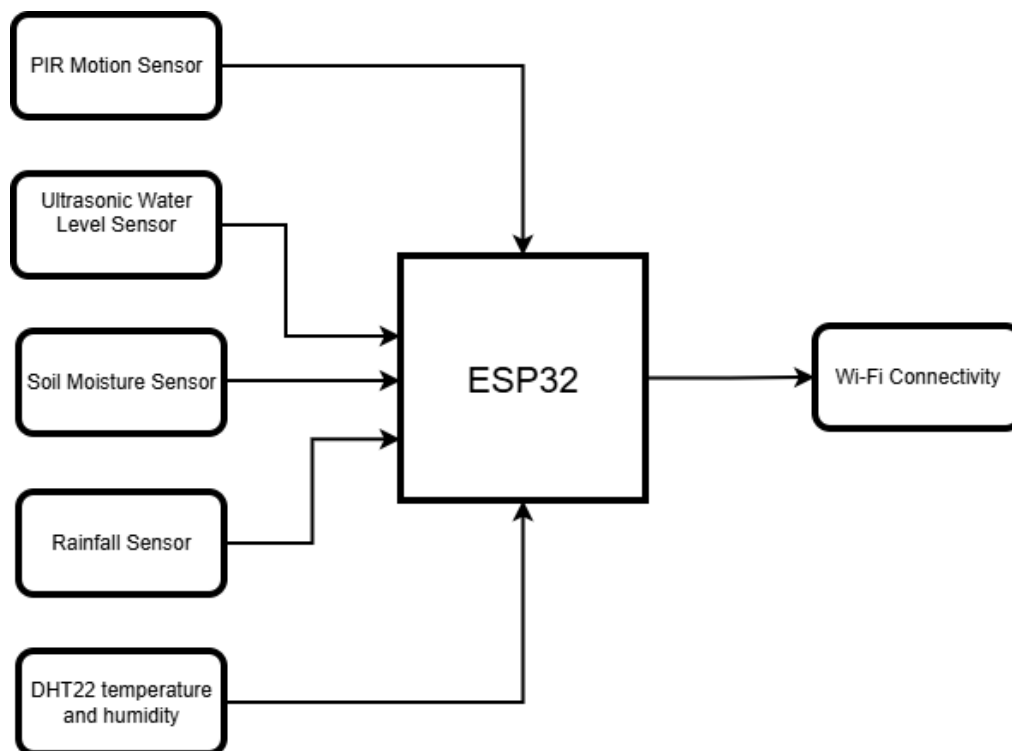


Figure 11: Hardware Design Block Diagram

3.2.1 Electronics Circuit Designs

3.2.1.1 Ultrasonic Water Level Sensor



Figure 12: Ultrasonic Sensor

In this application the ultrasonic sensor is implemented to measure the water level in the tank. The connection of the sensor to the ESP32 is illustrated in the figure 12 with the echo pin connected to GPIO pin 26, trigger pin connected to GPIO pin 25, VCC connected to GPIO pin 3.3V and GND connected to ground. The ultrasonic sensor measures the distance using sound waves, where a pulse is emitted by the trigger pin and travels through air then reflects off the surface of the object and then captured by the echo pin. The distance is calculated by considering the speed of sound and the time it takes for the wave to travel from trigger to echo. This sensor plays a critical role in ensuring that sufficient water is available before irrigation is activated, and allows farmers to monitor water usage.

3.2.1.2 Soil Moisture Sensor

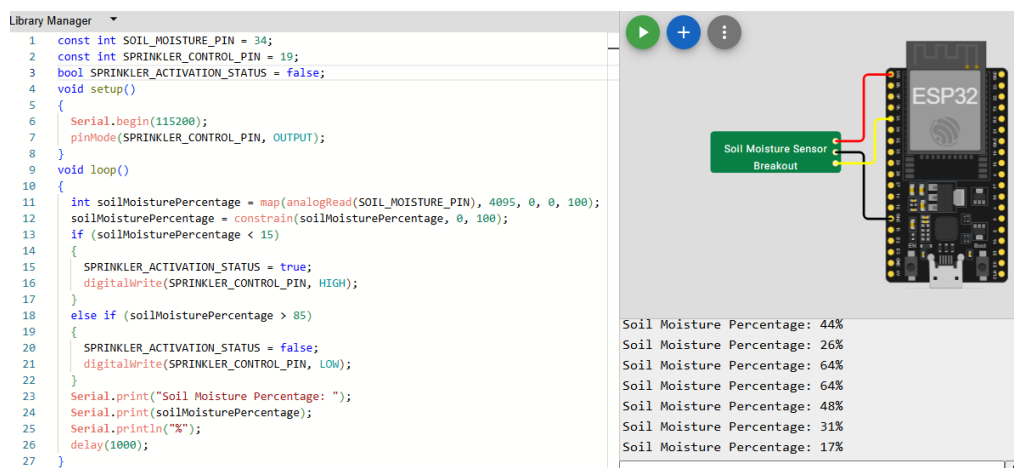


Figure 13: Soil Moisture Sensor Circuit Design

The soil moisture sensor is a key component that primarily defines the irrigation process. Figure 13 shows the circuit diagram of the soil moisture sensor, with the VCC pin connected to 3.3V, GND connected to ground pin, and the sensor output connected to analog GPIO pin 34. The

raw sensor values are mapped into percentage form ranging from 0% (dry soil) and 100%(saturated), for easier interpretation and monitoring. The status of this sensor is indicated using the serial monitor which displays the soil moisture percentage. This sensor is essential because it provides the input used for irrigation decision-making and also serves as feedback during irrigation control to prevent overwatering.

3.2.1.3 PIR Motion Sensor

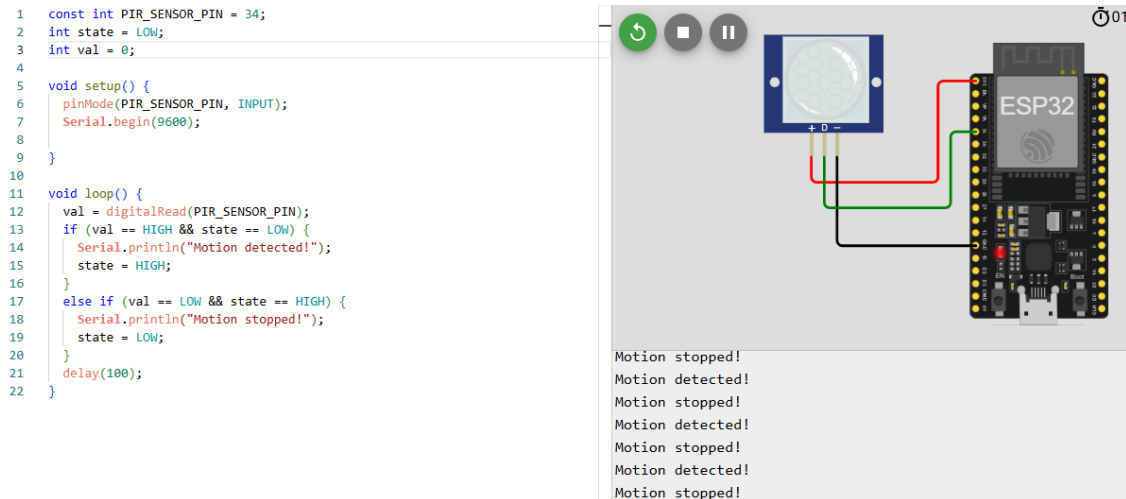


Figure 14: PIR Motion Sensor Circuit Design

The PIR motion sensor detects infrared radiation emitted by humans and animals, enabling the system to identify movement within the irrigation area. The sensor is powered using the ESP32 3.3V and ground at the positive and negative pins and the sensor output(D) is connected to GPIO pin 34. The sensor outputs a digital signal HIGH if motion is detected and LOW if no motion is detected. The status of this sensor is indicated in the serial monitor. This circuit is implemented to warn individuals in the field when irrigation is about to begin, improving safety and preventing water wastage.

3.2.1.4 DHT22 Temperature and Humidity Sensor

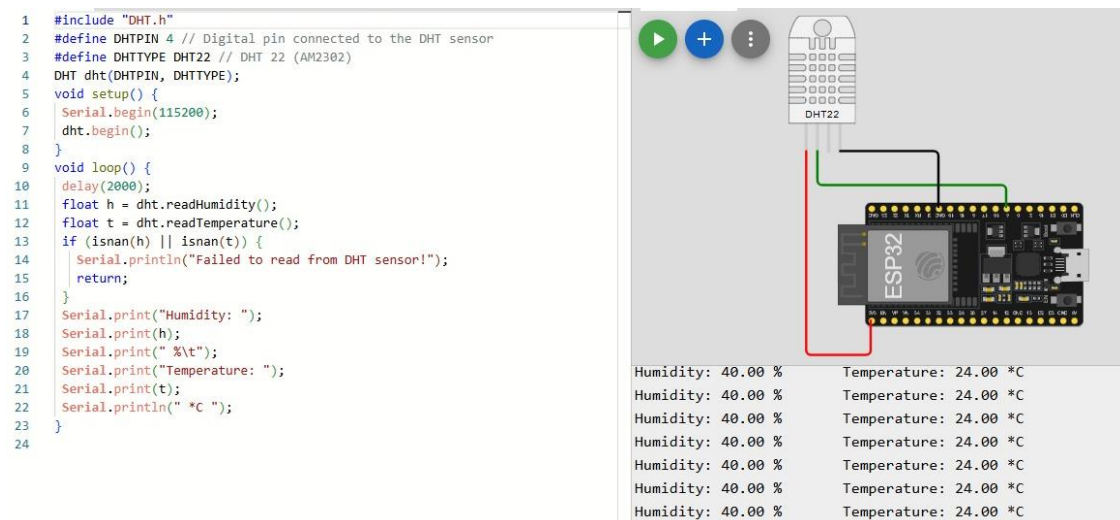


Figure 15: DHT22 Temperature and Humidity Sensor Circuit Design

The DHT22 sensor measures temperature and humidity of the farm environment to effectively manage soil, water and support irrigation decisions. Figure 15 shows the circuit diagram for the DHT22 integrated with the ESP32. The sensor output is connected to digital GPIO pin 4, VCC is connected to 3.3V pin and GND is connected to ground. It integrates with the ESP32 using the DHT.h library to accurately capture temperature and humidity which is mapped into percentage values. The status of the sensor at various threshold is indicated in the serial monitor. This sensor is used to monitor atmospheric conditions that influence soil moisture evaporation rates and crop water demand.

3.2.1.5 Rainfall Sensor Circuit Design

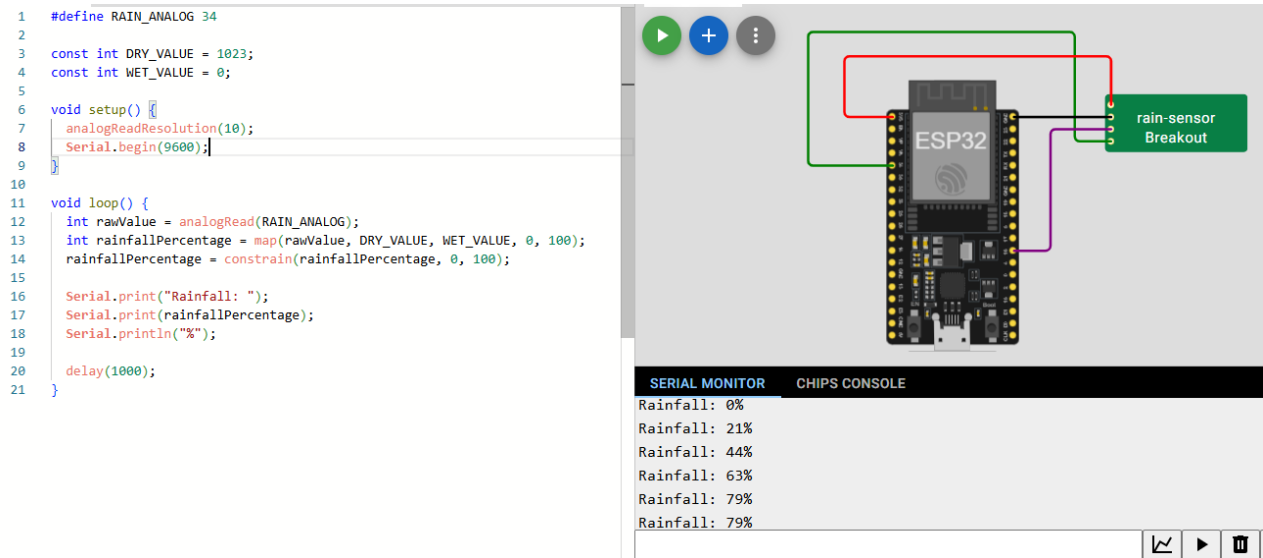


Figure 16: Rainfall Sensor Circuit Design

The rainfall sensor is connected to the ESP32 through an analog GPIO pin 34, and digital GPIO pin 16, VCC is connected to 3.3V and GND to ground. The ESP32 continuously monitors the reading of the rainfall pins and maps the reading into a percentage. These percentage reading is indicated in the serial monitor. The sensor module also includes a potentiometer that allows sensitivity adjustment of the detection threshold. This sensor is essential for preventing irrigation during rainfall, thereby ensuring efficiently water usage and avoiding over-irrigation.

3.3 Main Circuit Design

The main circuit integrates all individual sensor modules with the ESP32 microcontroller to form a complete smart irrigation system. Each sensor is connected to dedicated GPIO pins for data acquisition, while the ESP32 processes all incoming signals and executes irrigation control logic based on predefined conditions. The integrated system ensures coordinated operation between environmental sensing, decision-making, and irrigation control, resulting in an automated and efficient farming solution.

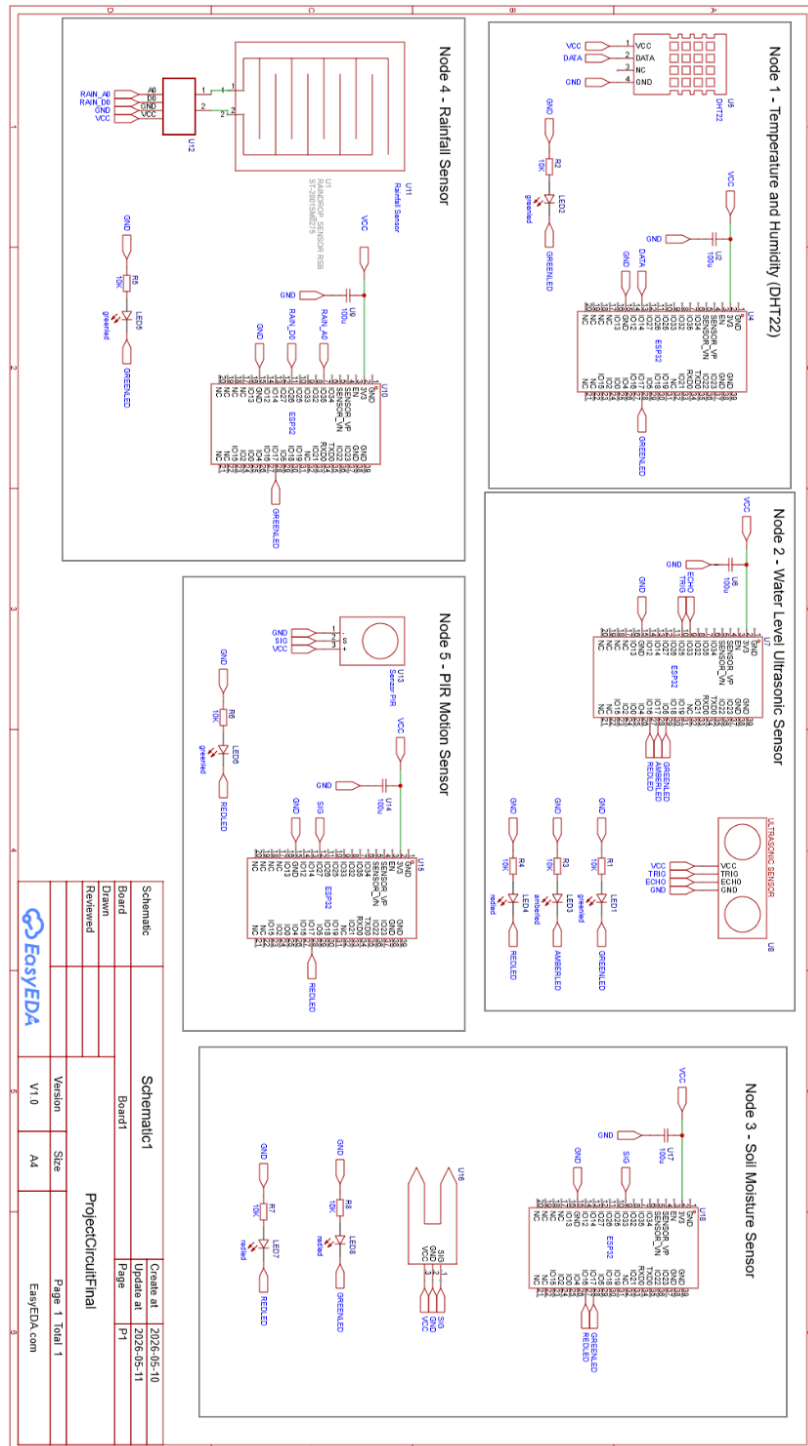


Figure 17: Main Circuit Design Sketch

3.4 Wiring

The system is wired using five ESP32 microcontroller nodes with each sensor connected to each microcontroller. Each sensor is connected individually to separate nodes to avoid pins conflict. To keep the wiring less complex and simples all ESP32 are powered using the 3.3V, 5V and ground pin reducing power failure disasters. Table 3 shows how each ESP32 node connects to each sensor. The physical connection for all ESP32 nodes integrated with each

sensor is illustrated in figure 18. Each ESP32 microcontroller is configured as static and connected to server Wi-Fi for sensor data transfer.

Table 3: Sensor to ESP32 Connection

Sensor Nodes	ESP32 Pins	Sensor Pins
Node 1: DHT22	3V3	VCC
	GND	GND
	GPIO 14	DATA
Node 2: Ultrasonic Sensor	3V3	VCC
	GND	GND
	GPIO 25	TRIG
	GPIO 26	ECHO
Node 3: Soil Moisture Sensor	3V3	VCC
	GND	GND
	GPIO 34	SIG
Node 4: Rainfall Sensor	3V3	VCC
	GND	GND
	GPIO 35	RAIN A0
	GPIO 26	RAIN D0
Node 5: PIR Motion Sensor	3V3	VCC
	GND	GND
	GPIO 27	SIG

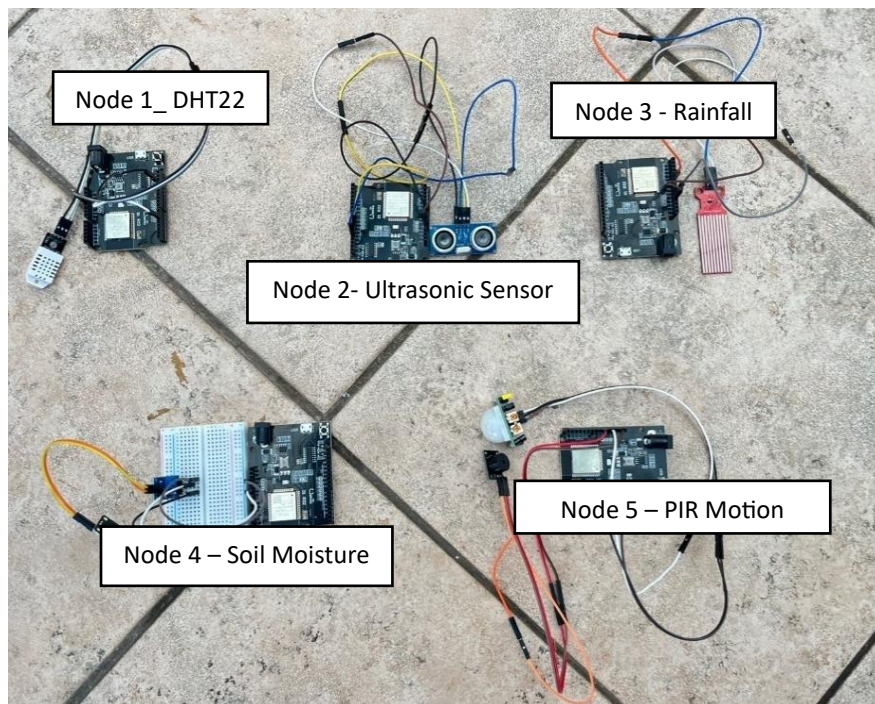


Figure 18: System Wiring

3.5 Enclosure Design

The final implementation of the circuit includes the complete circuit connection and the enclosure design. Each node is now connected into a breadboard including a circuit components and status indicators as shown in figure below. The enclosure design is made up of cardboard material to protect microcontrollers, breadboard. Additionally, a plastic container

mounted in the enclosure is used as farm land and a plastic bottle is to be used for the water tank. Each ESP32 and wires are mounted on the side of the enclosure to prevent tangling, physical damage and dust exposure. The status indicators and sensors are mounted at top of the enclosure using breadboard to accurately capture the farm environmental conditions and visualize the status of each sensor.

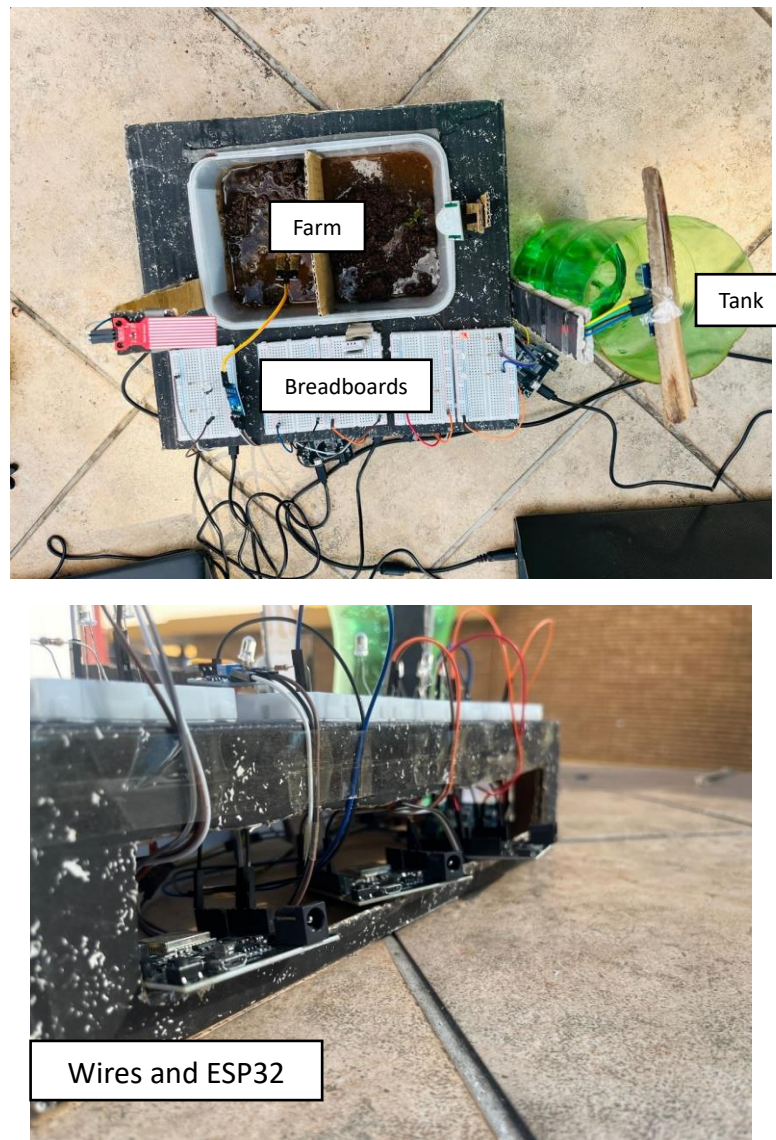


Figure 19: Enclosure Design

3.6 Safety Precautions

The ESP32 based monitoring and control system involves working electrical components, water-based elements and electrical wiring. When assembling and operating the system it is important to follow safety precautions. The following safety precautions were followed when assembling and operating the systems include the following:

- Connect components to the proper ESP32 voltages (5V or 3.3V) before powering the circuit to prevent permanent damage of microcontroller and sensor.

- Ensure components have no direct contact with water and soil since the system works with water and soil moisture.
- Ensuring that the power supply is disconnected before inserting or removing components from breadboard.
- Prevent overloading the GPIO pins on ESP32 to prevent current damage.
- The system must not be operated in extremely cold or hot conditions
- Ensure that the water tank does not have leaks to prevent abnormal water level readings and water from reaching components

3.7 Conclusion

This chapter presented the hardware design of the ESP32-based smart irrigation monitoring and control system. It described the overall hardware architecture and explained how each component, including the ESP32 microcontroller, sensors, are connected and integrated to form a complete system. A detailed circuit designs for the soil moisture sensor, ultrasonic water level sensor, PIR motion sensor, DHT22 temperature and humidity sensor, and rainfall sensor and nodes were discussed to show how environmental data is captured. Through this hardware design, a functional and integrated system capable of real-time environmental monitoring and automated irrigation control was established. The next chapter will focus on the software implementation of the system.

Chapter 4: Software Design of the Design and Implementation of ESP32 based Smart Irrigation Monitoring and Control System for Effective Soil-Water System in Farms

4.1 Introduction

In this section a detailed explanation of the software design implemented in the smart irrigation system for monitoring and controlling the soil and water conditions. The software design enables the integration of the hardware design described in chapter 3, allowing the system to monitor and control the system remotely. This data is analysed using web server and stored to database. To develop the software of the smart irrigation and monitoring system different analysis models are implemented which includes, use case diagram, activity diagram, physical data modelling.

4.2 Software Design

Figure 20 shows the block diagram of the overall software design of the system. The systems software consists of different components responsible for processing data collected by ESP32 and send over a Wi-Fi network.



Figure 20: Software Design Block Diagram

4.2.1. Use Case Analysis

A use case diagram is one of the design components that models the system requirements focusing on the functionality and the behaviour of the system (Rahman, Binti Hashim and Yusof, 2021). It considering the interaction between the actor which is the person or another system that interacts with the use case. Figure 21 shows the use case description of the software for the system. The system consists of four actors, farmer, administrator, Flask Server System, and MySQL Database System. Both the administrator and farmer can login to the system. The farmer can only access one sensor and the administrator can access all the sensors at the same time. The Flask Server System is responsible for receiving, transferring and executing logic on sensor data. MySQL Database stores sensor data and user credentials. The Web Server System is responsible for displaying sensor data. The use case diagram provides a clear analysis of the overall system functionality making it easier to visualize and scale.

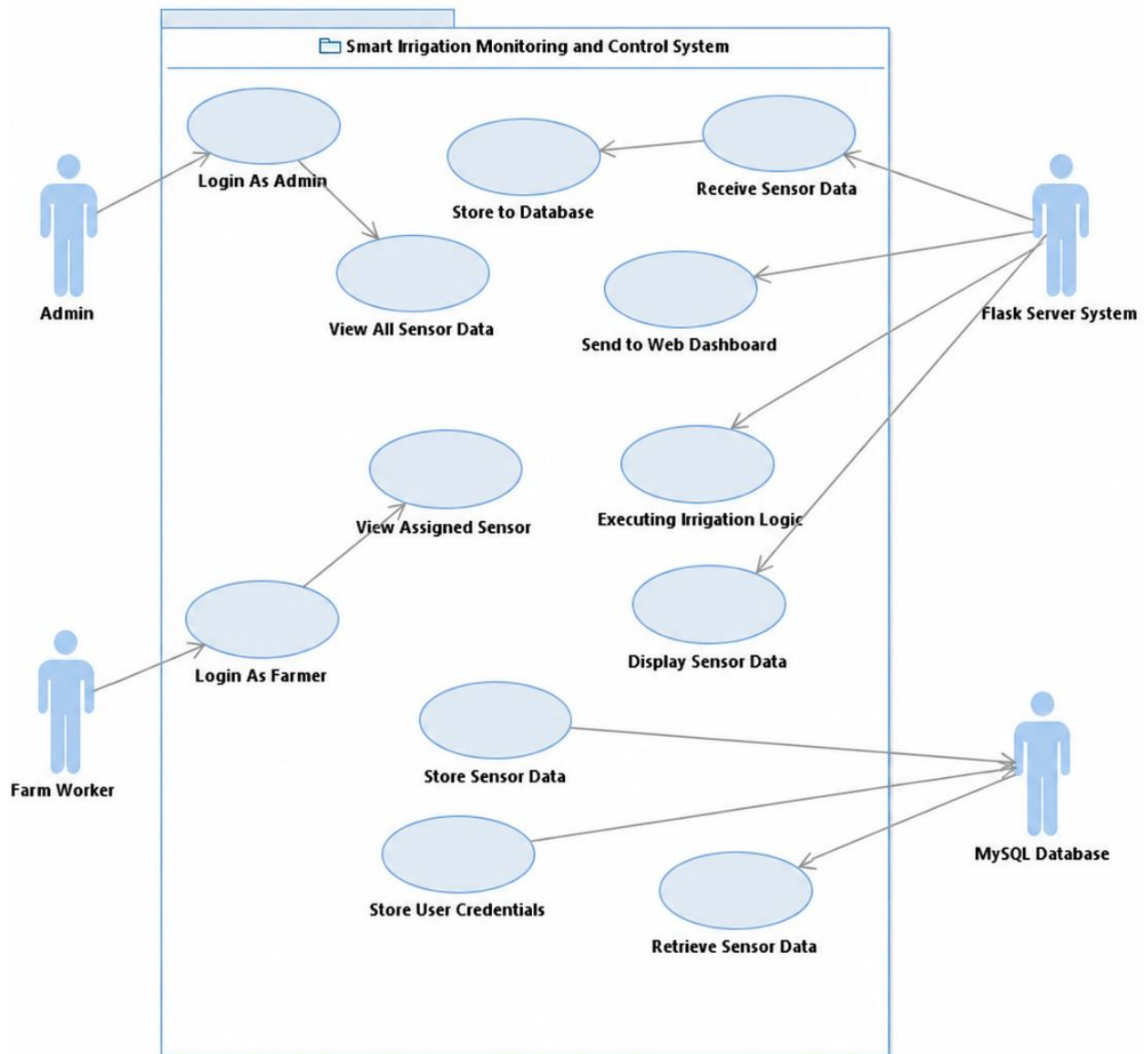


Figure 21: Use Case Diagram

4.2.1.1 Activity Diagram – Execute Irrigation Logic Use Case

An activity diagram is a design model that describes the complex operations, workflow and logic of the system (S. Al-Fedaghi, 2021). For this project the activity diagram is implemented to describe the logic of the irrigation process by considering the sequence of actions and decisions taken before the system can irrigate. Figure 22 shows how the Flask Server System Actor mentioned in figure 20 performs the Execute Irrigation Logic Use-Case.

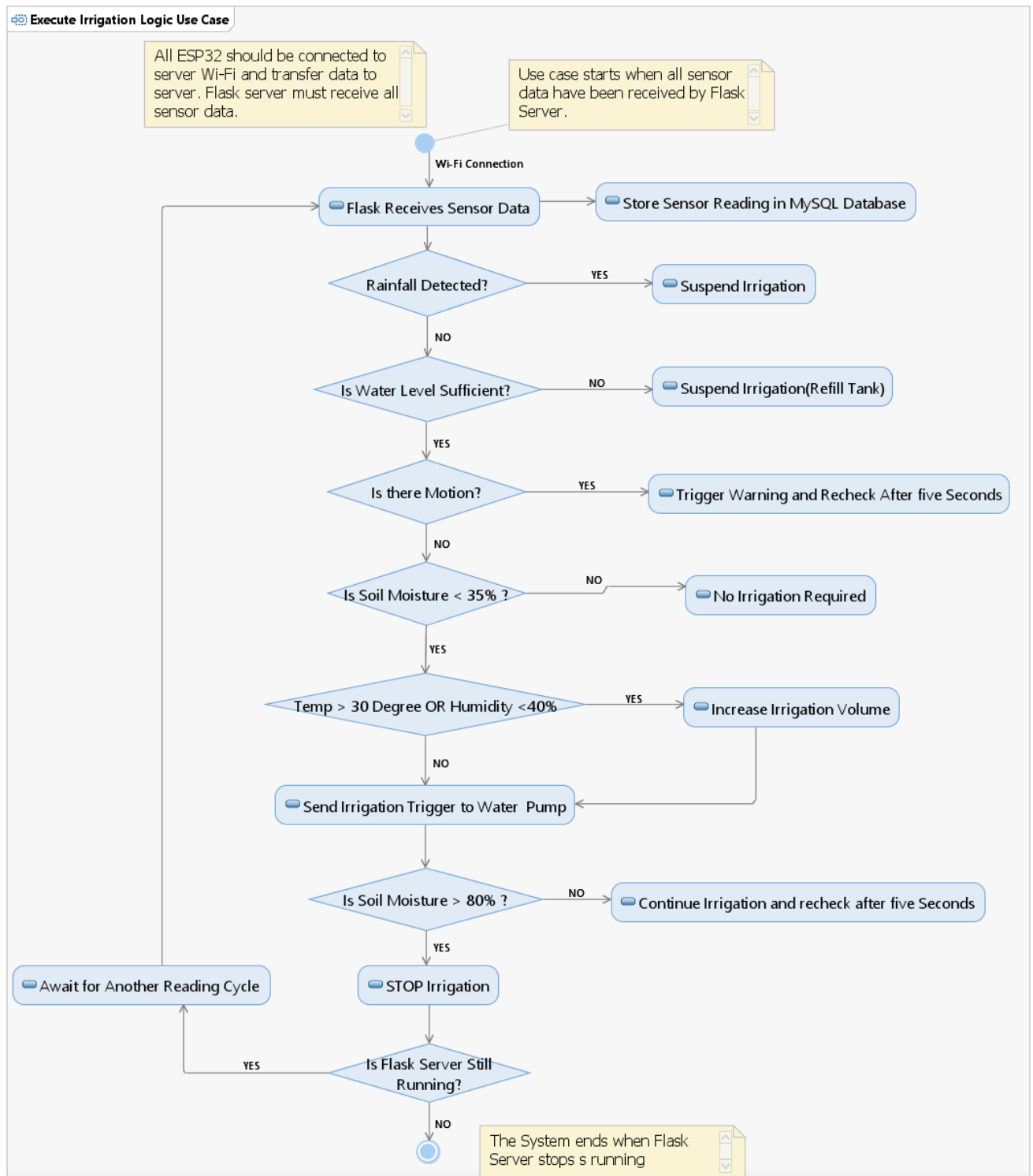


Figure 22: Activity Diagram

4.2.1.2 Sequence Diagram

A sequence diagram is a model that describes the sequential interaction between actors and the system over time. It uses life lines and the exchanges of messages between actors and the system. Figure 23 shows the sequence diagram which describes the steps taken by users when interacting with the system focusing on the website navigation. In the diagram both

users (administrator and farmer) submit their credentials using the login for authentication. The flask server validates the credentials by comparing them with the database values. Upon successfully authenticating the farmer is routed to their dedicated sensor page and the administrator is routed to the administrator dashboard.

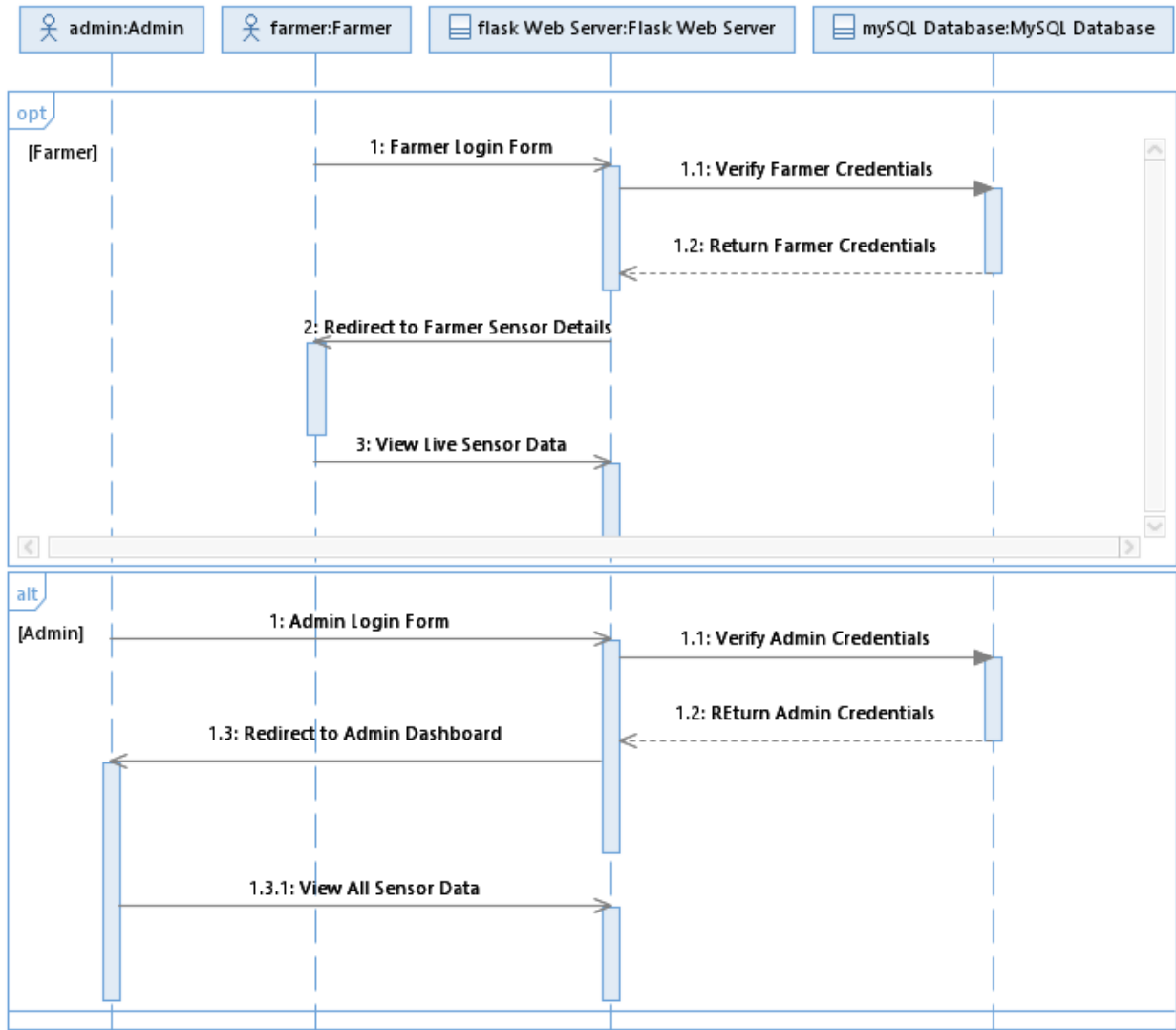


Figure 23: Sequence Diagram

4.2.2. Flask Server

4.2.2.1 Wi-Fi Connectivity and Data Transfer

The flask server acts as a gateway that facilitates communication between ESP32, database and web interface. It establishes a client-server relationship with ESP32 using Wi-Fi. The host PC acts as an access point to allow the ESP32 client to connect. Figure 24 shows the ESP32 program used to setup the Wi-Fi connectivity with the machine that runs the Flask server.

```

#include <WiFi.h>

const char* ssid = "HuaweiPC";
const char* password = "Refilwe06";

void setup() {
    Serial.begin(115200);
    WiFi.begin(ssid, password);

    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\nConnected!");
    Serial.println(WiFi.localIP());
}

```

Figure 24: ESP32 Wi-Fi Setup

Upon establishing Wi-Fi connectivity with the Flask server, the ESP32 sends HTTP packets to the server over the Wi-Fi connection. Flask server uses HTTP POST/GET methods to capture packets. Figure 25 shows the code used to capture sensor reading from ESP32. The request contains a JSON payload that contains sensor_id and sensor data. These are extracted using request.get_json() function and stored as dictionary.

```

220 @app.route('/data', methods=['POST'])
221 def receive_data():
222     global current_session_id
223
224     data = request.get_json()
225     if not data or 'id' not in data:
226         return jsonify({"status": "error", "message": "Invalid data"}), 400
227
228     sensor_id = data['id']
229     sensor_data[sensor_id] = data
230     print(f"[RECEIVED] sensor_id={sensor_id} | data={data}")
231
232     if current_session_id is None:
233         current_session_id = create_session()
234         if current_session_id is None:
235             return jsonify({"status": "error", "message": "No session"}), 500
236

```

Figure 25: Capturing ESP32 Sensor Data

4.2.2.2 Database Connection and Communication

Flask server communicates with the database using a MySQL connector that is installed and imported as seen in figure 26. When flask runs a session is created that connects to database pool. The database is hosted locally using port 3306.

```

10  from mysql.connector import pooling
11
12  db_pool = pooling.MySQLConnectionPool(
13      pool_name="smartpool",
14      pool_size=10,
15      host="localhost",
16      user="root",
17      port=3306,
18      password="B@nele2002",
19      database="SmartIrrigationDB",
20      auth_plugin="mysql_native_password"
21  )

```

Figure 26: Flask and Database Connection

Upon establishing connection between flask and MySQL database, flask can start sending sensor data to the database table using queries. Figure 27 shows an illustration of how data is inserted into the database.

```

223  def receive_data():
224      if sensor_id == "soil":
225          value = data.get("distance")
226          if value is not None:
227              cursor.execute(
228                  "INSERT INTO SoilMoisture (SessionID, Moisture) VALUES (%s, %s)",
229                  (sid, float(value)))
230              db.commit()
231              print(f"[DB] SoilMoisture saved: {value}")
232
233      elif sensor_id == "rainfall":
234          raw = data.get("value"); status = data.get("status")
235          if raw is not None and status is not None:
236              cursor.execute(
237                  "INSERT INTO Rainfall (SessionID, RawValue, RainStatus) VALUES (%s, %s, %s)",
238                  (sid, int(raw), str(status)))
239              db.commit()
240              print(f"[DB] Rainfall saved: raw={raw}, status={status}")
241
242      elif sensor_id == "temp":
243          value = data.get("distance")
244          if value is not None:
245              cursor.execute(
246                  "INSERT INTO Temperature (SessionID, TempValue) VALUES (%s, %s)",
247                  (sid, float(value)))
248              db.commit()
249              print(f"[DB] Temperature saved: {value}")
250
251

```

Figure 27: Sending to Database

4.2.3. Database Modelling

When designing a database model several developments steps are taken to achieve the final model. Firstly, a conceptual model is developed followed by a logical model, then the physical data model. The physical data model is one of the key aspects of data modelling which includes how data will be stored and how it will be accessed (Šušter and Ranisavljević, 2023). Figure 28 shows the physical model implemented for this system which consists of eight tables. Below is the breakdown of each table:

- The SoilMoisture table record the values of the soil moisture sensor and stores them as a decimal value.
- The Temperature table stores temperature reading captured from the DHT22 in decimal format using Degrees Celsius.
- The Humidity table stores the humidity part of the DHT22 in percentages
- The Rainfall table stores the ADC value of the rainfall sensor along with the status which indicates whether it is raining or not
- The motion table stores the PIR sensor results using characters as a status that indicates whether motion was detected or not.
- The water level table stores the tank water level in decimal which represent the percentage.
- The user table stores credentials of administrator and farmers for authentication and authorization in the web server.
- The session table generated a SessionID which links all the sensor tables together. Whenever the flask server restarts a new session is created. To avoid database flooding, deleting a session deletes all sensor values collected during that session.

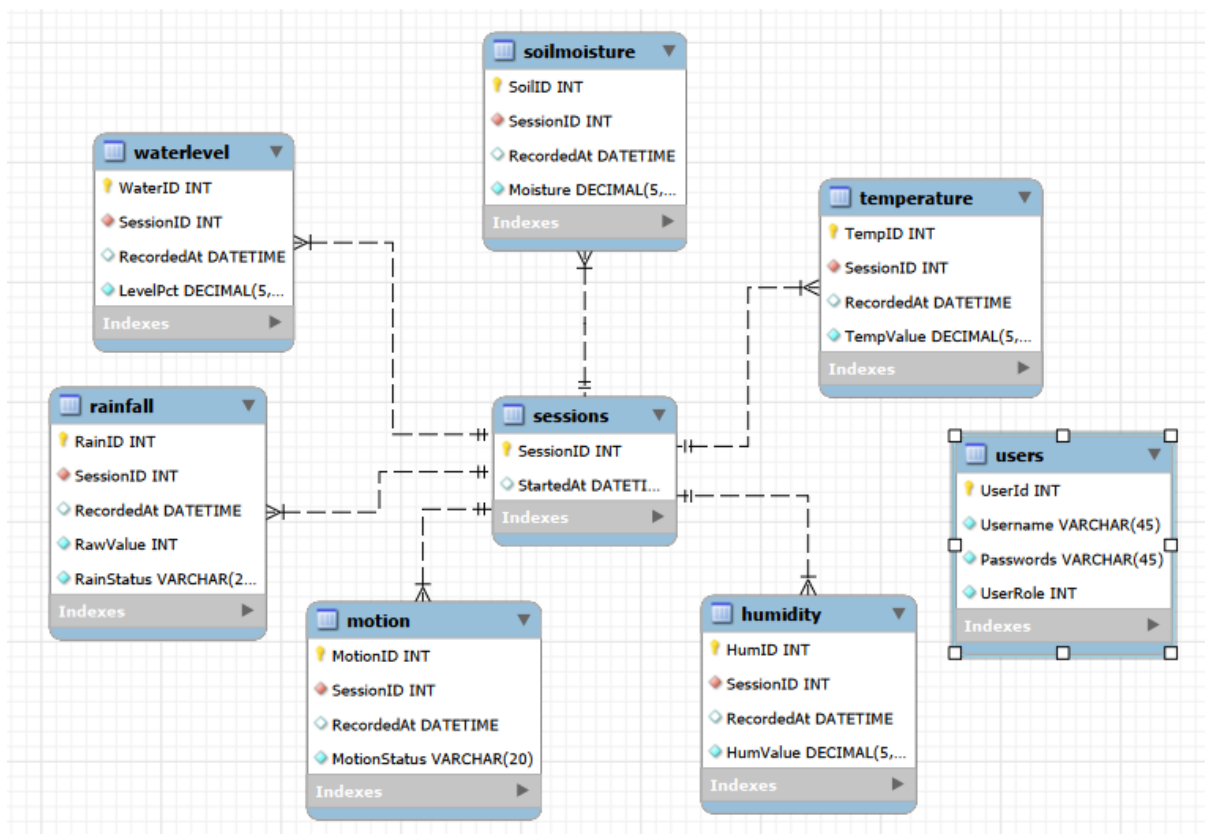


Figure 28: Physical Database Model

4.2.4. Web Interface

4.2.4.1 Home Page

The administrator has the privilege of monitoring and controlling all sensor data. Figure 29 illustrates the homepage of the website. In the homepage the user has an option to login as the farmer or the administrator.

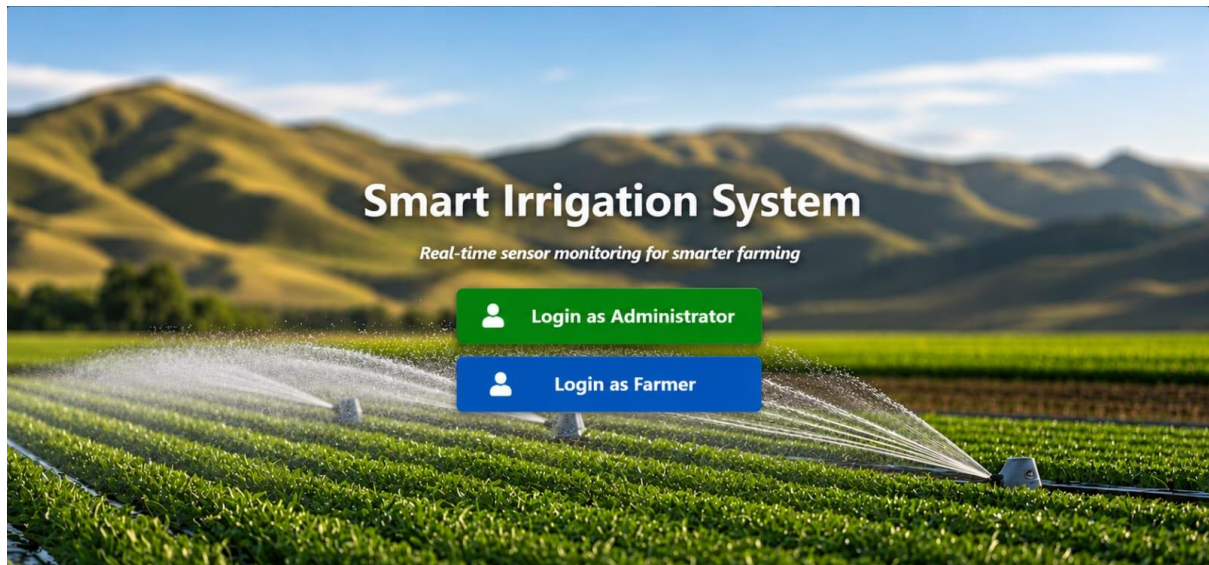


Figure 29: Home Page

4.2.4.2 Login Page

The web server consists of a login page which allows users to login as either administrator or farmer using. Figure 30 shows the administrator login page, where the administrator uses the username and password to login. Figure 31 shows farmer login page, where the farmer the username, password and the sensor role.

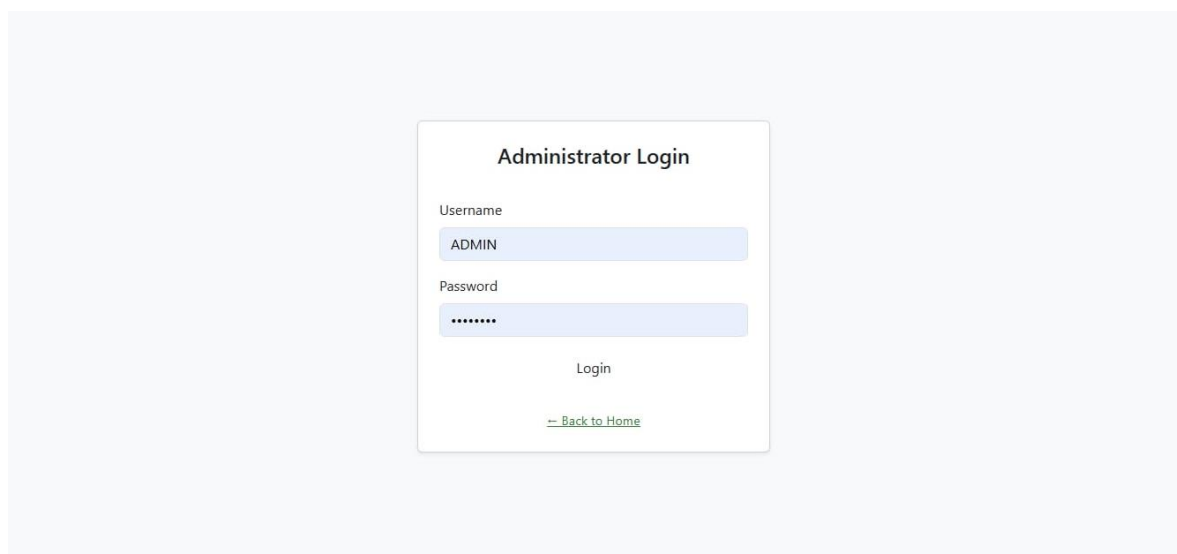
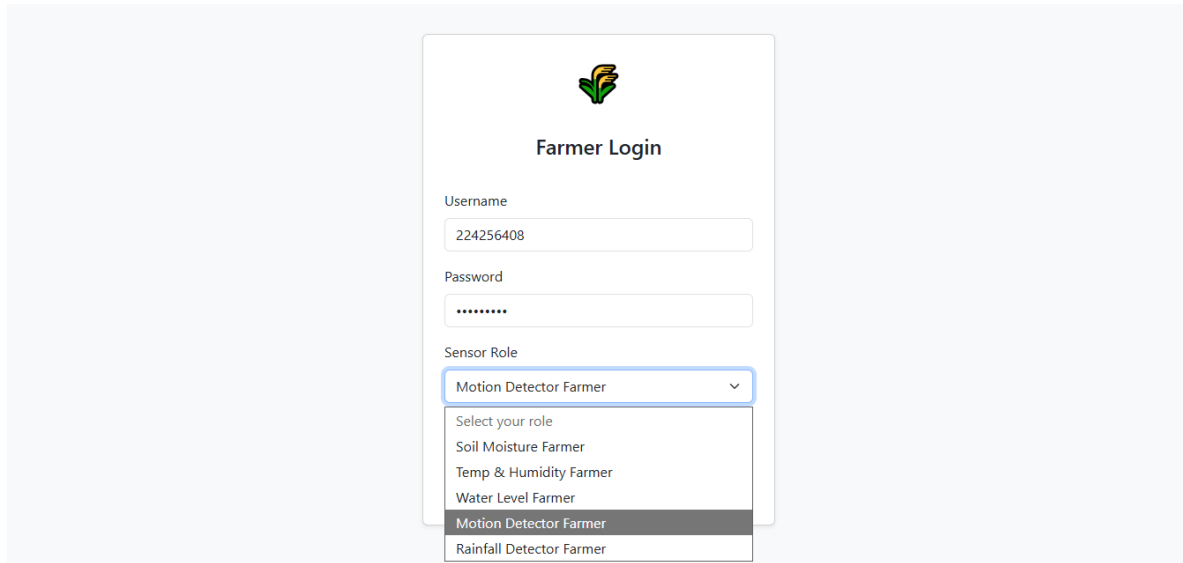
The image displays a web form titled 'Administrator Login'. The form is contained within a light gray box. It features two input fields: 'Username' with the text 'ADMIN' entered, and 'Password' with a masked input of seven asterisks. Below these fields is a 'Login' button. At the bottom of the form, there is a link that reads '< Back to Home'. The entire form is set against a light gray background.

Figure 30: Administrator Login Page

The image shows a web form titled "Farmer Login" with a small plant icon at the top. It contains three input fields: "Username" with the value "224256408", "Password" with masked characters "*****", and a "Sensor Role" dropdown menu. The dropdown is open, showing a list of roles: "Select your role", "Soil Moisture Farmer", "Temp & Humidity Farmer", "Water Level Farmer", "Motion Detector Farmer" (which is highlighted), and "Rainfall Detector Farmer".

Farmer Login

Username
224256408

Password

Sensor Role
Motion Detector Farmer

Select your role
Soil Moisture Farmer
Temp & Humidity Farmer
Water Level Farmer
Motion Detector Farmer
Rainfall Detector Farmer

Figure 31: Farmer Login Page

4.3 Conclusion

In this chapter the software design strategies and tools for the ESP32-based smart irrigation monitoring and control system was presented. The overall software architecture of the system describes how the system monitors environmental conditions to automate the irrigation process using the flask server, database and webserver illustrated by the block diagram in figure 20. The software design includes the use-case analysis diagram, activity diagram, and database design. A brief description of the client-server Wi-Fi connection, database connection, data transfer, and data storage was provided. The flask server is the most significant component in the entire software design with the responsibility of ensuring that data is received, stored and dispersed to database and web-server. The next chapter will present the results and discussions and system performance analysis based on sensor reading, database output and system behaviour at different environmental conditions.

Chapter 5: Results and Discussions

5.1 Introduction

In this chapter the results obtained during the design and implementation of the smart irrigation and monitoring system will be documented. This includes real-time sensor reading, database records and data visualization using graphs obtained for each sensor data over a specific time. The results drawn are directly from the testing of the entire integrated system, which incorporates both the hardware and software design outlined in Chapter 3 and 4. The results obtained are essential to assess the system design, performance, and behaviour at different environmental conditions.

5.2 Findings

When constructing the project several problems were encountered and resolved. These problems and solution are listed below.

- During the operation of the system some nodes would lose Wi-Fi connection to the server due to high traffic of data transfer happening at the same time. To address this a logic to transfer data at different intervals was used.
- When data is transferred to flask server some sensors values would be lost during the process which causes the database to store NULL values. To prevent this, additional logic was added to store data only when the values are received from the ESP32.
- During testing the wires would disconnect due to a small vibration or touch each other which led to short-circuiting. To counteract this, jumper wires were securely mounted on and routed along the edge of the enclosure.
- The system inserts data of five sensors to five different tables which flooded the host device storage after the system is left running for a long time, which led to slow query performance over time. To address this an SQL query that drops old rows after a certain time was defined.

5.3 Results

5.3.1 Sensor Data

In this project the ultrasonic sensor is implemented as the water level sensor, by measuring the distance from the sensor to the water surface. Figure 32 shows the web interface that displays real-time water level percentage from the irrigation tank and the graph of the last 20 water level reading every minute. When the water level is above 80 % a “Full” signal is displayed, when the water level is between 50% and 80% the “Medium” signal and when water level is below 50% the “Low” signal is displayed.

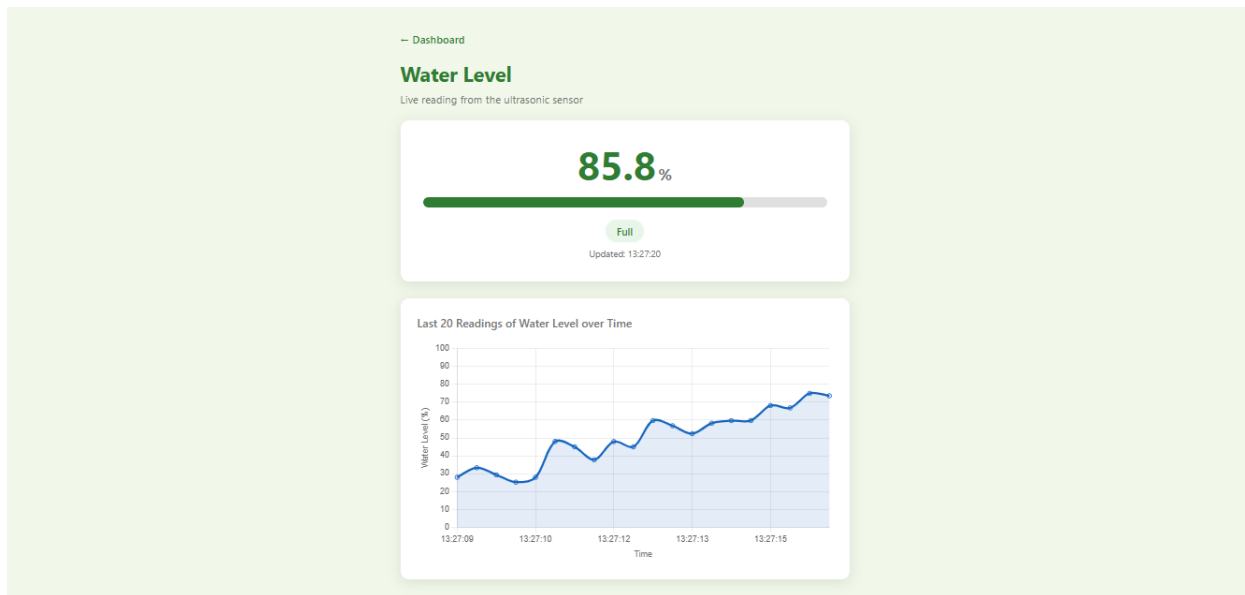


Figure 32: Water Level Readings

The displayed data corresponds directly with the records stored in the database. Figure 33 shows the database values of the water level sensor at different percentages at different times. The database stores the timestamp and the values captured at that time.

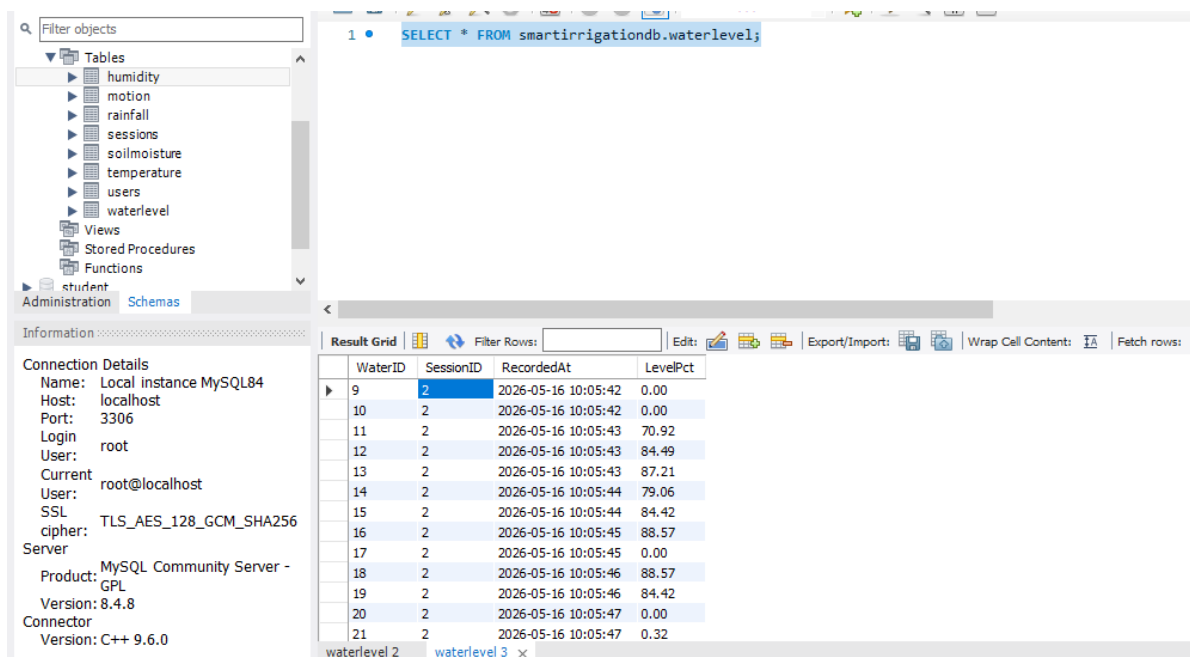


Figure 33: Water Level Database Values

5.2.1 Administrator Results

In the system the administrator has the privilege to view all the sensor data. Figure 34 shows the administrator dashboard which display all live sensor data and the irrigation status which is controlled by a logic based on different sensor thresholds. When the administrator presses the "View Details" they can access each sensor data individually and monitor the real-time data and visualize the data in a graph for the last 20 reading of the sensor as seen in figure 35.

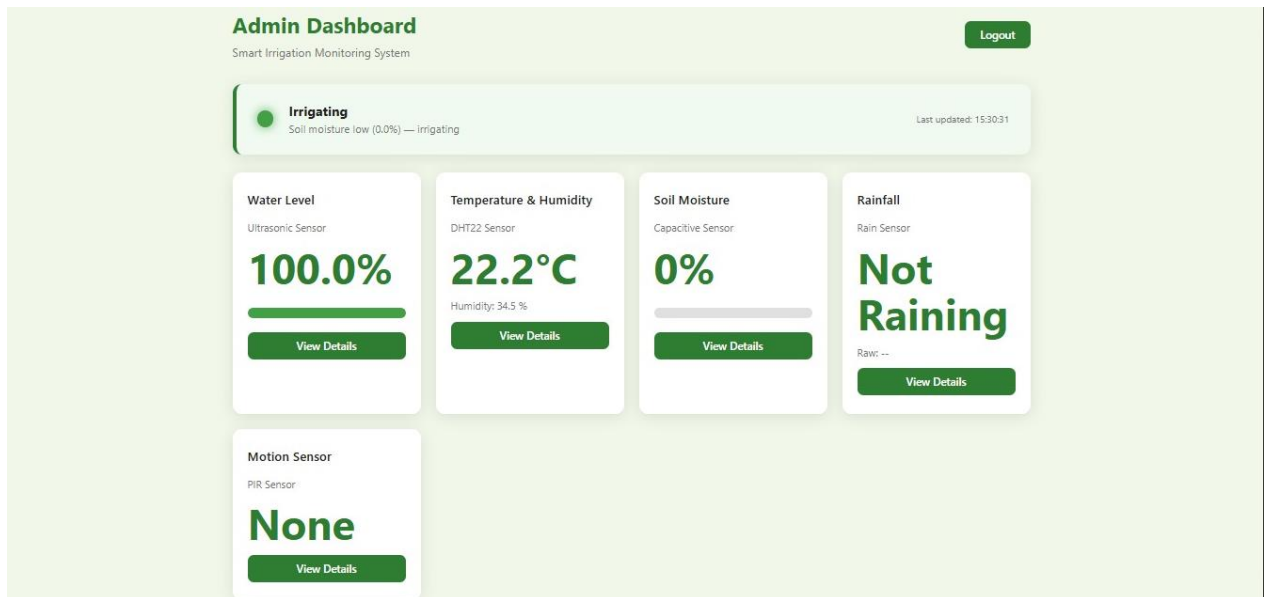


Figure 34: Administrator Dashboard

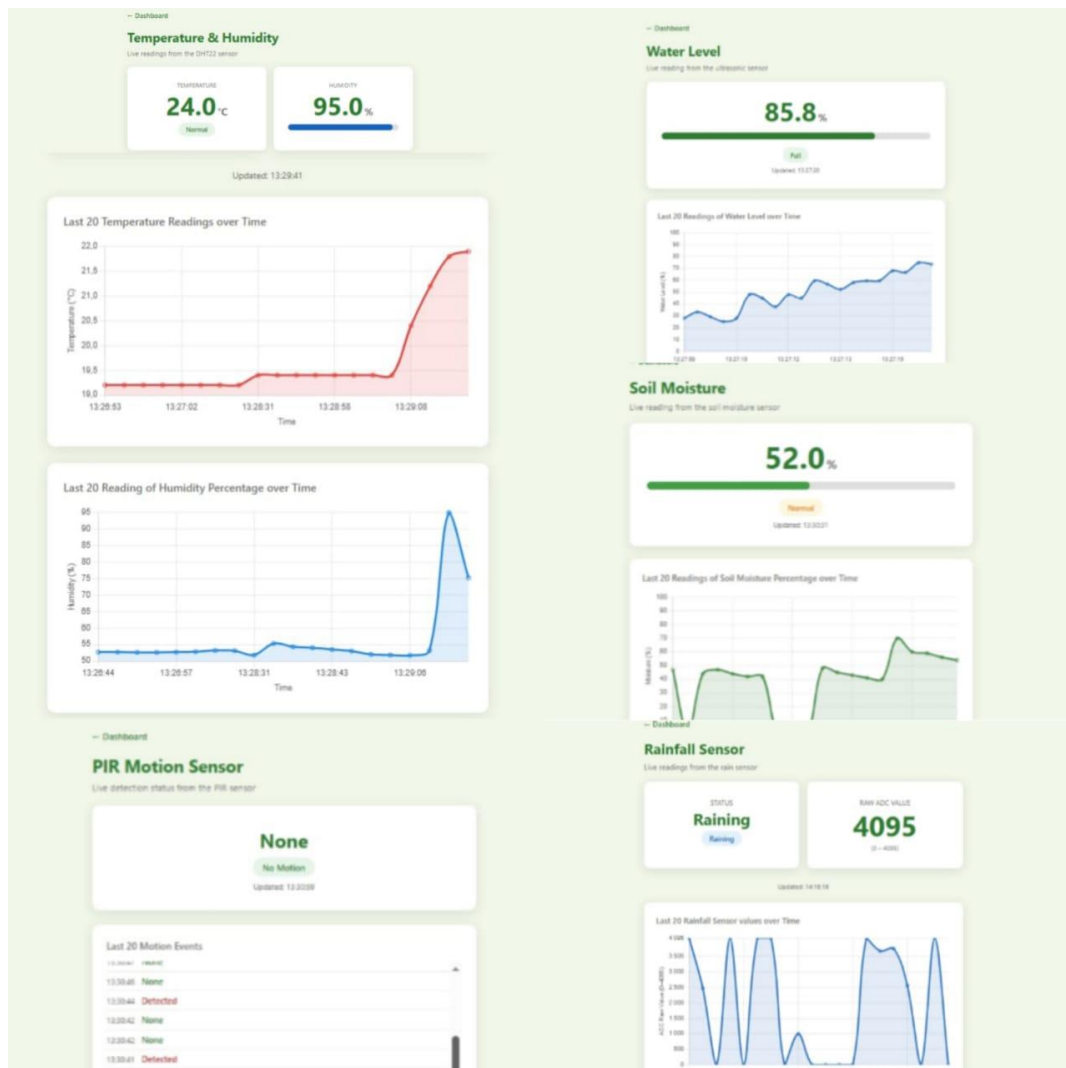


Figure 35: Sensor Data from Administrator View

Chapter 6: Conclusion and Recommendations

6.1 Conclusion

The objectives of this project were successfully achieved in monitoring and controlling irrigation for effective soil and water management. The hardware design of the project was efficiently implemented using five sensors which were integrated with five microcontrollers for monitoring environmental condition of in the farm. The hardware components integrated effectively with the software design which included the Flask server acting as a gateway between the hardware and software using Wi-Fi connectivity. All sensor data was routed correctly to the web-server through Flask and stored successfully in the database using MySQL.

6.2 Recommendations

Upon designing and implementing the system, several recommendations and future improvements are proposed to improve the overall system design. These include:

- Integrating the system with a cloud platform like AWS or Google cloud to allow farmers to access the system from anywhere using the internet.
- Saving energy by replacing grid electricity with solar power which is reliable and cost-efficient.
- Integrating machine learning which can be trained on historical data to predict patterns and support better irrigation decisions.
- Implementing a weather API that will provide upfront rainfall forecasts rather than depend only on data captured by the sensor.

References

Anon, (n.d.). Available at:

https://www.mantech.co.za/datasheets/products/170689_BG.pdf?srsId=AfmBOoqC0R2qVMVcy-35F0N2mKW7uTYm0jLWJuzyeOKI9D2MjXfKzWfg.

[Accessed: 23 April 2026]

Chowdhury, S., Sen, S. and Janardhanan, S. (2022). Comparative Analysis and Calibration of Low Cost Resistive and Capacitive Soil Moisture Sensor. Available at:

<https://arxiv.org/pdf/2210.03019> [Accessed: 23 April 2026]

Djalilov, A., Sobirov, E., Nazarov, O., Urolov, S. and Gayipov, I. (2023). Study on automatic water level detection process using ultrasonic sensor. 1142(1), pp.012020–012020. Available at: <https://doi.org/10.1088/1755-1315/1142/1/012020>.

[Accessed: 24 March 2026]

ElecFreaks (2011). Ultrasonic Ranging Module HC-SR04. [online] Available at:

<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>.

[Accessed: 23 April 2026]

Fan, S. (2021). The advantages and disadvantages of agricultural economic management information. Journal of Physics: Conference Series, 1744(3), p.032020. Available at:

<https://doi.org/10.1088/1742-6596/1744/3/032020>.

[Accessed: 27 March 2026]

Al-Fedaghi, S. (2021). (IJACSA) International Journal of Advanced Computer Science and Applications. Computer Engineering Department Kuwait University Kuwait. [online]

Available at: <https://arxiv.org/pdf/2106.16160>

[Accessed: 7 May 2026]

Fity.club. (2026). Available at: <https://fity.club/lists/w/wemos-d1-r32>

[Accessed 20 March 2026].

Ghimire, D. (2020). Comparative study on Python web frameworks: Flask and Django.

[online] Available at:

https://www.theseus.fi/bitstream/handle/10024/339796/Ghimire_Devndra.pdf?sequence=2&isAllowed=y.

[Accessed: 9 April 2026]

Gridling, G. and Weiss, B. (2007). Introduction to Microcontrollers. [online] Available at:

<http://hopfen.topfen.net/uni/mc/MC-Things/Microcontroller.pdf>.

[Accessed: 29 April 2026]

Handson Technology User Guide HC-SR501 Passive Infrared (PIR) Motion Sensor. (n.d.).

Available at: <https://www.handsontec.com/dataspecs/SR501%20Motion%20Sensor.pdf>.

[Accessed: 24 April 2026]

Harris, D. and Fuller, D. (2014). Agriculture: Definition and Overview. [online]

ResearchGate. Available at:

https://www.researchgate.net/publication/301345493_Agriculture_Definition_and_Overview [Accessed: 24 March 2026]

Hussain, S. and Wani, I. (2024). JETIR2404D40 Journal of Emerging Technologies and Innovative Research, 11(4). Available at: <https://www.jetir.org/papers/JETIR2404D40.pdf>. [Accessed: 15 April 2026]

Liu, T. (n.d.). Digital-output relative humidity & temperature sensor/module DHT22 (DHT22 also named as AM2302) Capacitive-type humidity and temperature module/sensor. [online] Available at: <https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf>. [Accessed: 23 April 2026]

Pereira, G.P., Chaari, M.Z. and Daroge, F. (2023). IoT-Enabled Smart Drip Irrigation System Using ESP32. IoT, 4(3), pp.221–243. Available at: <https://doi.org/10.3390/iot4030012>. [Accessed: 25 March 2026]

Rahman, S.A., Binti Hashim, W. and Yusof, A. (2021). Designing A Use Case Diagram For Developing An Electricity Consumption (EC) System. [online] IEEE Xplore. Available at: <https://doi.org/10.1109/ICCOINS49721.2021.9497156>. [Accessed: 5 May 2026]

Sumathi, M.S., Kelagadi, H.M., Akanksha, A., Navyashree, A., Rakshitha and Sriha, D. (2022). Security System Using PIR Sensors. 2022 International Interdisciplinary Humanitarian Conference for Sustainability (IIHC), pp.1525–1529. Available at: <https://doi.org/10.1109/iihc55949.2022.10060701>. [Accessed: 6 April 2026]

Šušter, I. and Ranisavljević, T. (2023). Optimization of MySQL database. Journal of Process Management. New Technologies, 11(1-2), pp.141–151. Available at: <https://doi.org/10.5937/jouproman2301141q>. [Accessed: 2 May 2026]

Wallace, J.S. (2000). Increasing agricultural water use efficiency to meet future food production. Agriculture, Ecosystems & Environment, 82(1-3), pp.105–119. Available at: [https://doi.org/10.1016/s0167-8809\(00\)00220-6](https://doi.org/10.1016/s0167-8809(00)00220-6). [Accessed: 5 April 2026]

Wardani, I.K., Ichniarsyah, A.N., Telaumbanua, M., Priyonggo, B., Fil'aini, R., Mufidah, Z. and Dewangga, D.A. (2023). The feasibility study: Accuracy and precision of DHT 22 in measuring the temperature and humidity in the greenhouse. IOP Conference Series, 1230(1), pp.012146–012146. Available at: <https://doi.org/10.1088/1755-1315/1230/1/012146>. [Accessed: 23 March 2026]